

LAPORAN PRAKTIKUM MOBILE PROGRAMMING
(Navigasi Antar Halaman dengan Mengirimkan Argumen Menggunakan
Named Route)



Nama : Dimas Bagus Hari Murti

NIM : 240605110080

Kelas : B

Tanggal : 8 September 2025

JURUSAN TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM
MALANG
GANJIL 2025/2026

I. Tujuan

Tujuan dari percobaan ini adalah untuk memahami dan mengimplementasikan navigasi antar halaman pada aplikasi Flutter menggunakan konsep Named Route. Selain itu, praktikum ini juga bertujuan agar mahasiswa mampu mengirimkan data atau argumen dari satu halaman ke halaman lain, serta menampilkannya secara dinamis di halaman tujuan. Melalui kegiatan ini, diharapkan mahasiswa dapat memahami cara kerja pengiriman data menggunakan *arguments* dan mengaitkannya dengan tampilan UI, khususnya dalam konteks menampilkan detail data game yang diambil dari API FreeToGame.

II. Langkah Kerja

- Membuat project Flutter baru dengan nama navigasi_argumen.
- Menambahkan file home.dart, tujuan.dart, dan screen_arguments.dart ke dalam folder lib.
- Mengatur route di main.dart menggunakan Named Route dan menetapkan halaman awal Home.
- Membuat class ScreenArguments untuk mengirim data antar halaman.
- Mengambil data game dari API FreeToGame di halaman Home dan menampilkannya di UI.
- Menambahkan navigasi dari halaman Home ke halaman Tujuan dengan membawa data melalui argument.
- Menampilkan detail data game di halaman Tujuan sesuai data yang diterima dari Home.

III. Screenshot Hasil

1. Source code:

- main.dart

```
import 'package:flutter/material.dart';
import 'home.dart';
import 'tujuan.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF0081C9)),
        useMaterial3: true,
      ),
      initialRoute: '/',
      routes: {
        '/': (context) => const Home(),
        Tujuan.routeName: (context) => const Tujuan(),
      },
    );
  }
}
```

Kode di atas digunakan untuk mengatur struktur dasar aplikasi Flutter dengan sistem navigasi antar halaman. Pada bagian `MaterialApp`, saya menonaktifkan debug banner, menambahkan tema warna utama, serta menentukan halaman awal (`initialRoute`) yaitu `'/'` yang mengarah ke class `Home`. Selain itu, di bagian `routes`, saya mendefinisikan dua rute yaitu `'/'` untuk halaman `Home` dan `Tujuan.routeName` untuk halaman `Tujuan`. Dengan pengaturan ini, aplikasi bisa berpindah antar halaman menggunakan `Named Route`, sehingga kode navigasinya jadi lebih rapi dan mudah dikelola.

- home.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'screen_arguments.dart';
import 'tujian.dart';

class Home extends StatefulWidget {
  const Home({super.key});

  @override
  State<Home> createState() => _HomeState();
}

class _HomeState extends State<Home> {
  var title, thumbnail, shortDesc, description;
  var genre, platform, release, cover, gameid, publisher;

  Future getGame(String gameid) async {
    http.Response response = await http.get(
      Uri.parse('https://www.freetogame.com/api/game?id=$gameid'),
    );
    var results = jsonDecode(response.body);
    setState(() {
      this.gameid = gameid;
      title = results['title'];
      thumbnail = results['thumbnail'];
      shortDesc = results['short_description'];
      description = results['description'];
      genre = results['genre'];
      platform = results['platform'];
      publisher = results['publisher'];
      release = results['release_date'];
      cover = results['screenshots'][0]['image'];
    });
  }

  @override
  void initState() {
    super.initState();
    getGame('475');
  }
}
```

```
23 @override
24 Widget build(BuildContext context) {
25   return Scaffold(
26     backgroundColor: const Color(0xFF081c9),
27     body: SafeArea(
28       child: Center(
29         child: gameid == null
30           ? const CircularProgressIndicator()
31           : Column(
32             mainAxisAlignment: MainAxisAlignment.min,
33             children: [
34               GestureDetector(
35                 onTap: () {
36                   Navigator.pushNamed(
37                     context,
38                     Tujian.routeName,
39                     arguments: ScreenArguments(
40                       cover,
41                       title,
42                       description,
43                       shortDesc,
44                     ), ScreenArguments
45                   );
46                 },
47               ),
48               child: Container(
49                 padding: const EdgeInsets.all(15),
50                 margin: const EdgeInsets.all(15),
51                 decoration: BoxDecoration(
52                   color: Colors.white,
53                   borderRadius: BorderRadius.circular(15),
54                 ),
55               ),
56               child: Column(
57                 children: [
58                   Image.network(thumbnail),
59                   const SizedBox(height: 15),
60                   Text(
61                     title,
62                     style: const TextStyle(
63                       fontSize: 24,
64                       fontWeight: FontWeight.bold,
65                     ),
66                   ),
67                   Text(
68                     description,
69                     style: const TextStyle(
70                       color: Colors.white,
71                       fontSize: 16,
72                     ),
73                   ),
74                   Text(
75                     shortDesc,
76                     style: const TextStyle(
77                       color: Colors.white,
78                       fontSize: 16,
79                     ),
80                   ),
81                   Text(
82                     publisher,
83                     style: const TextStyle(
84                       color: Colors.white,
85                       fontSize: 16,
86                     ),
87                   ),
88                   Text(
89                     release,
90                     style: const TextStyle(
91                       color: Colors.white,
92                       fontSize: 16,
93                     ),
94                   ),
95                   Text(
96                     genre,
97                     style: const TextStyle(
98                       color: Colors.white,
99                       fontSize: 16,
100                    ),
101                  ),
102                ],
103              ),
104            ],
105          ),
106        ),
107      ),
108    ),
109  );
110 }
```

```
const SizedBox(height: 10),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceAround,
  children: [
    Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text('Genre: $genre'),
        Text('Platform: $platform'),
      ],
    ),
    Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text('Publisher: $publisher'),
        Text('Release: $release'),
      ],
    ),
  ],
), Row(
  ),
), Column(
  ), Container(
    ), GestureDetector(
  ),
), Column(
  ), Center(
  ), SafeArea(
  ), Scaffold(
)
}
```

Kode di atas digunakan untuk menampilkan halaman Home pada aplikasi Flutter yang berisi informasi tentang satu game dari API FreeToGame. Pada bagian `getGame()`, data game seperti judul, genre, platform, dan deskripsi diambil dari API menggunakan package `http`, lalu hasilnya di-decode dengan `jsonDecode`. Data yang sudah diambil disimpan ke dalam variabel dan ditampilkan di layar. Tampilan game dibuat menggunakan widget `Container` yang berisi gambar, teks, dan detail informasi. Widget ini dibungkus dengan `GestureDetector`, sehingga ketika ditekan, aplikasi akan berpindah ke halaman Tujuan dengan membawa data game melalui class `ScreenArguments`. Dengan cara ini, pengguna bisa melihat detail game yang lebih lengkap di halaman berikutnya.


```

85         ), TextStyle
86       ), Text
87     ],
88     ), Row
89   ), Container
90 ), GestureDetector
91 ],
92 ), Column
93 ), Container
94 ), SingleChildScrollView
95 ), SafeArea
96 ); Scaffold
97 }
98 }

```

Kode di atas merupakan tampilan halaman Tujuan yang berfungsi untuk menampilkan detail game yang sudah dikirim dari halaman Home. Data yang diterima berasal dari ScreenArguments, lalu diambil menggunakan `ModalRoute.of(context)!.settings.arguments`. Di dalam halaman ini, data seperti cover, judul, dan deskripsi game ditampilkan dalam tampilan yang lebih rapi menggunakan widget Container dan Column. Selain itu, terdapat tombol "Kembali ke game" yang dibuat dengan GestureDetector dan `Navigator.pop(context)` agar pengguna bisa kembali ke halaman Home. Secara keseluruhan, halaman ini berperan sebagai tampilan detail dari data yang dikirim melalui navigasi dengan Named Route.

- screen_arguments.dart

```

1 class ScreenArguments {
2   final String cover;
3   final String title;
4   final String description;
5   final String shortDesc;
6
7   ScreenArguments(this.cover, this.title, this.description, this.shortDesc);
8 }
9

```

Kode di atas berisi class ScreenArguments yang berfungsi sebagai model data untuk menyimpan informasi yang akan dikirim antar halaman. Class ini memiliki empat properti utama yaitu cover, title, description, dan shortDesc yang masing-masing menyimpan data gambar, judul, deskripsi panjang, dan deskripsi singkat dari sebuah game.

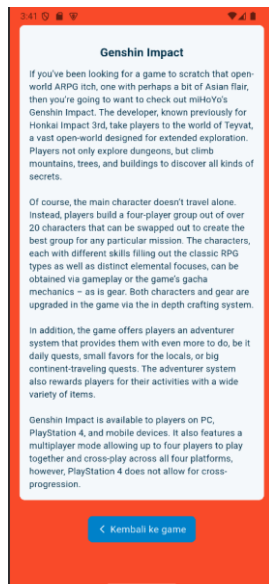
Dengan adanya class ini, proses pengiriman data antar halaman jadi lebih terstruktur dan mudah, karena semua informasi dikemas dalam satu objek sebelum dikirim melalui Named Route.

Output:

- Halaman Home



- Halaman Tujuan



IV. Kesimpulan

Dari percobaan ini, saya belajar bagaimana cara menggunakan Named Route untuk melakukan navigasi antar halaman pada aplikasi Flutter. Selain itu, saya juga memahami cara mengirim dan menerima data antar halaman menggunakan class `ScreenArguments` dan properti `arguments` pada `Navigator.pushNamed`. Dengan penerapan ini, aplikasi menjadi lebih terstruktur karena setiap halaman punya fungsi masing-masing dan data bisa ditampilkan secara dinamis. Praktikum ini juga memberikan pemahaman bahwa pengambilan data dari API dan menampilkannya di UI dapat dikombinasikan dengan sistem navigasi agar aplikasi terlihat lebih interaktif dan menarik.